JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 05

Lists, tuples, dictionaries and sets

```
learn / type / run / explain
```

Plain meaning and technical explanation

PLAIN MEANING

Collections keep related values together.

TECHNICAL EXPLANATION

Lists are ordered and mutable; tuples are ordered and immutable. Dictionaries map keys to values. Sets hold unique values.

REAL-WORLD USE

Represent post records, result lists and unique authors.

QUICK TIP

Use dictionary keys to explain each value in a record.

INSTALL / RUN

No packages. Windows: py workbook.py. macOS: python3 workbook.py.

Worked example 1: A list of titles

workbook.py

```
titles = ["Hello", "Learning"]
titles.append("Launch")
print(titles[0])
print(titles[-1])
print(len(titles))
```

PRO TIP Accessing an index outside the list raises IndexError.

Run and explain example 1

EXPECTED RESULT

Hello
Launch
3

WHY IT BEHAVES THIS WAY

Lists preserve insertion order. Index zero is the first item and -1 is the last.

Worked example 2: A post dictionary

workbook.py

```
post = {"id": "p1", "title": "Hello"}
post["title"] = "Updated"
print(post["title"])
print(post.get("image_path"))
print("title" in post)
```

PRO TIP post["missing"] raises KeyError.

Run and explain example 2

EXPECTED RESULT

Updated
None
True

WHY IT BEHAVES THIS WAY

Keys identify fields. get() returns None for a missing key unless a default is supplied.

Worked example 3: Tuples and unique authors

workbook.py

```
01    dimensions = (640, 480)
02    width, height = dimensions
03    authors = {"Ada", "Lin", "Ada"}
04    print(width, height)
05    print(sorted(authors))
```

PRO TIP Do not rely on set iteration order.

Run and explain example 3

EXPECTED RESULT

640 480
['Ada', 'Lin']

WHY IT BEHAVES THIS WAY

Tuple unpacking assigns both dimensions. A set removes duplicates; sorted() makes display order predictable.

Worked example 4: Copies and shared references

workbook.py

```
post = {"title": "Original"}
alias = post
copy = post.copy()
alias["title"] = "Changed"
print(post["title"])
print(copy["title"])
```

PRO TIP A shallow copy still shares nested mutable values.

Run and explain example 4

EXPECTED RESULT

Changed
Original

WHY IT BEHAVES THIS WAY

alias refers to the same dictionary. copy() creates a separate outer dictionary.

Exercise 1: Post collection

YOUR TASK

Create two post dictionaries in a list and update only the second title.

CHECK THESE CASES

List length stays two. The first title stays Hello. Second becomes Revised.

HINT AFTER A TRY

Index the list, then access the dictionary key.

Exercise 2: Unique formats

YOUR TASK

Count unique supported formats from a list containing duplicates.

CHECK THESE CASES

PNG, JPEG, PNG, WEBP has three unique formats. Original list remains unchanged.

HINT AFTER A TRY

Convert to a set and use len().

Checkpoint before moving on

01 Run and explain all four worked examples. Match their actual results to the expected behavior.

02 Attempt both exercises before reading the solutions. Verify every acceptance case.

03 Explain how this chapter supports Postlight: Represent post records, result lists and unique authors.

Official reference